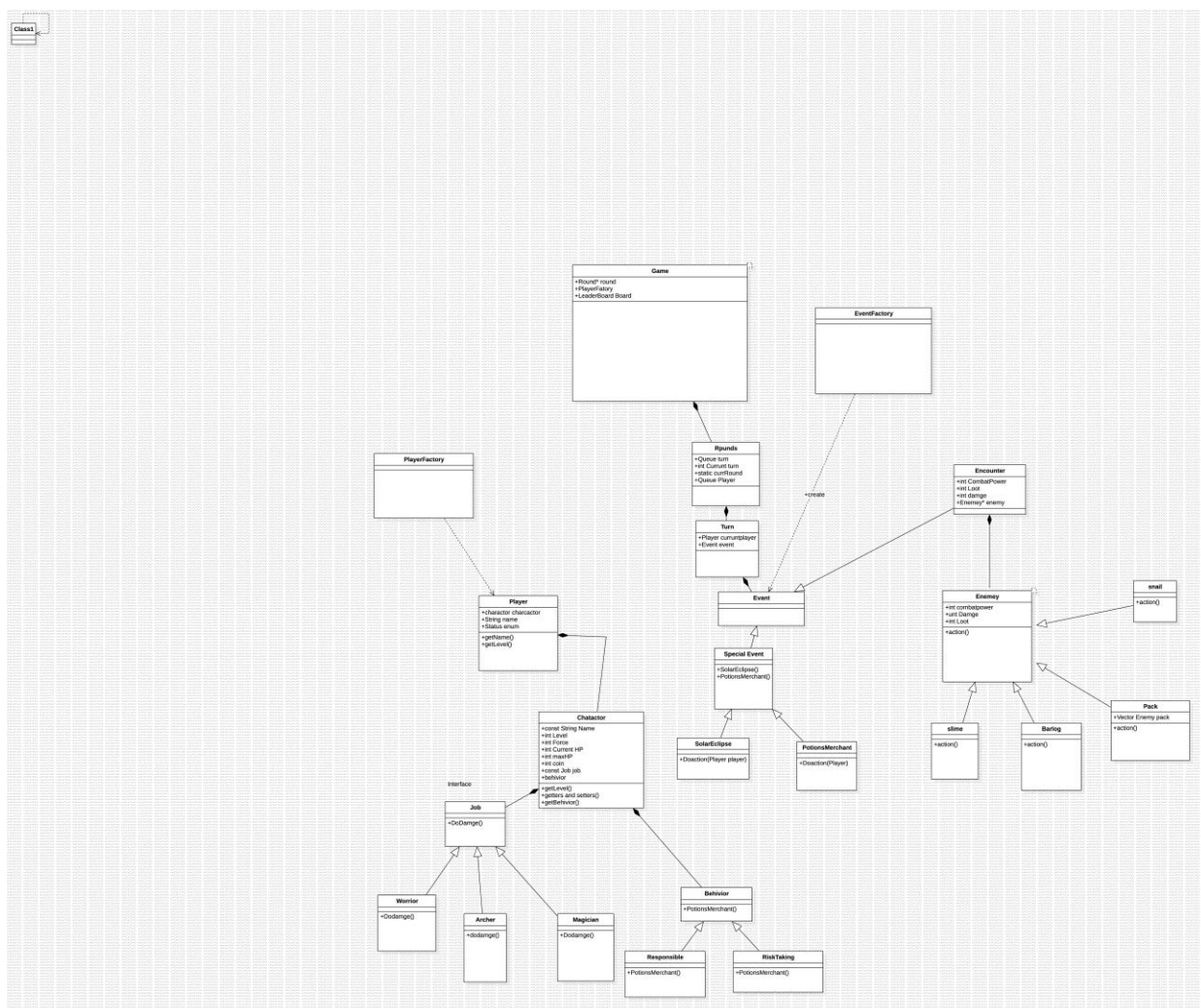




חלק יבש:

(1) סרטוט uml :



(2) שימוש ב design patterns :

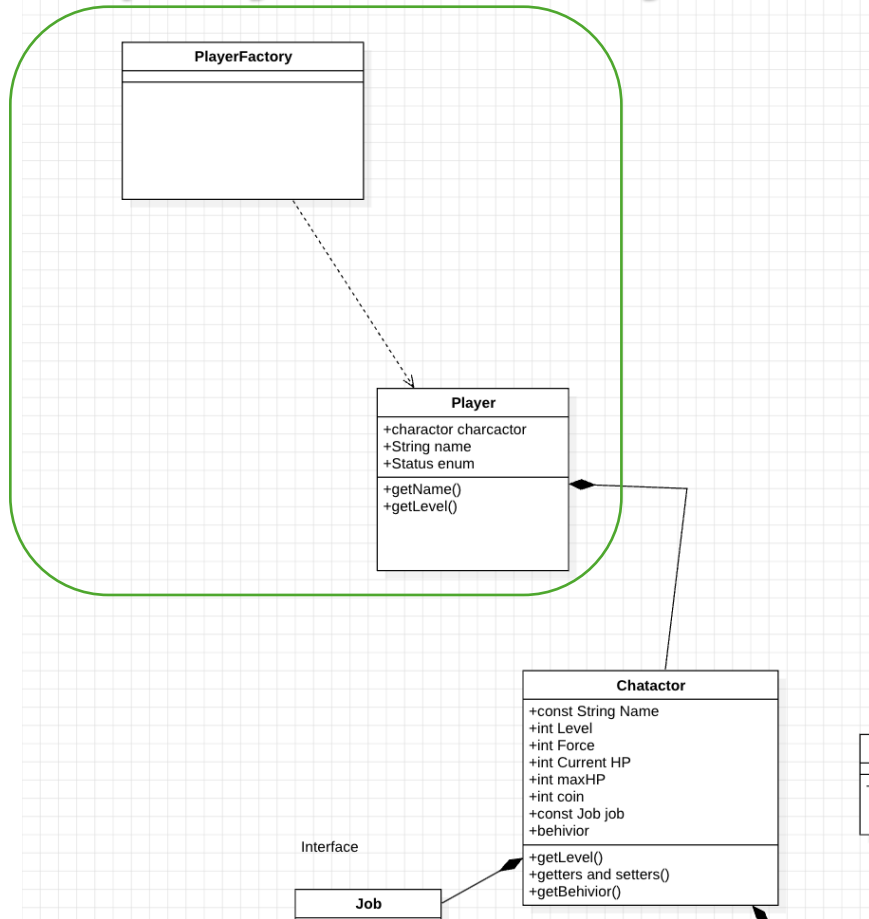
השתמשנו בהרבה סוגים מ תבניות עיצוב :

creational patterns עם סוג

השתמשנו ב סוג factory: בשני מקמות שונים:

*מקום ראשון מפעל של שחקנים הוא ש אחראי על יצור שחקנים חדשים

playerFactory

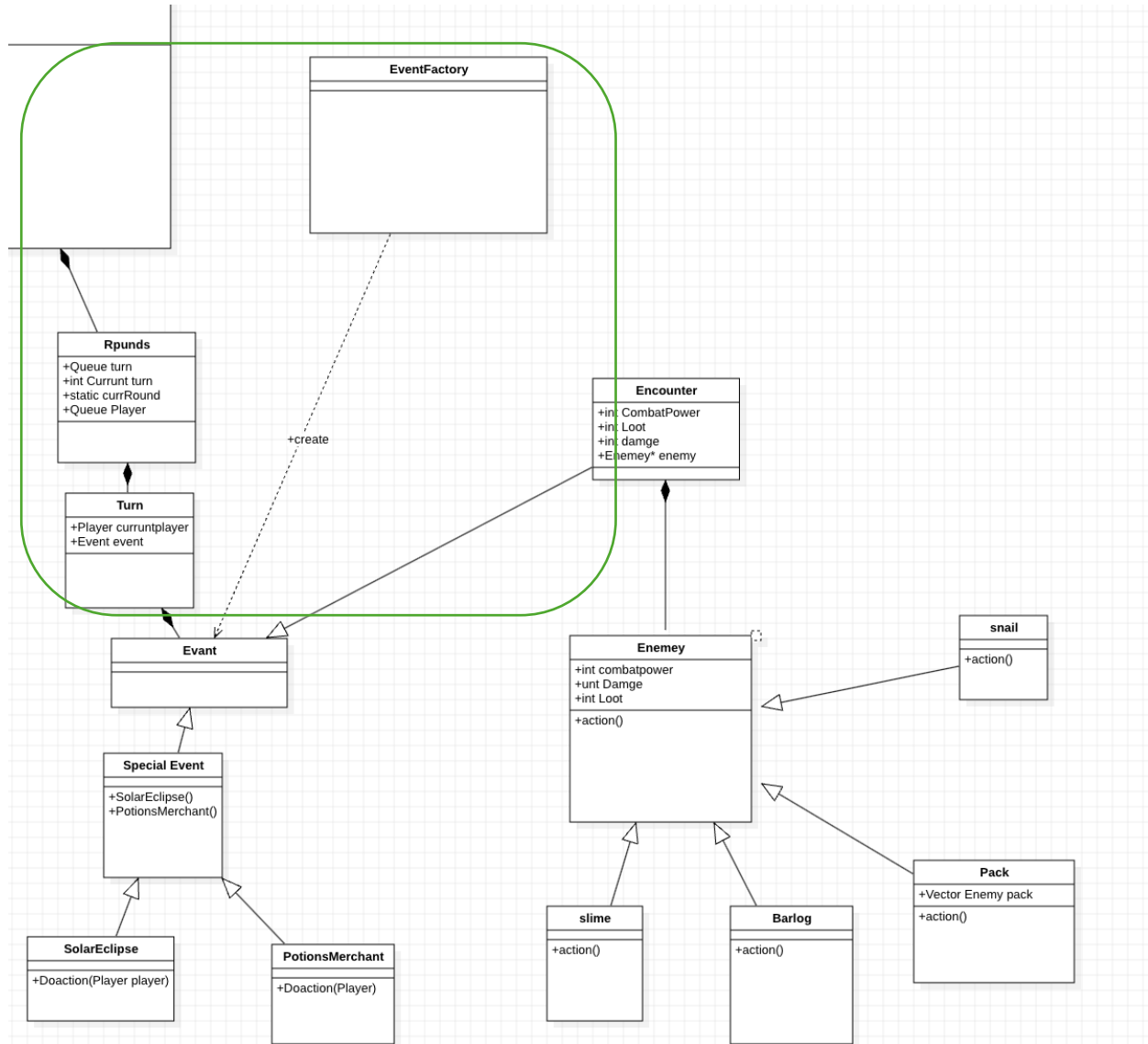


*מקרה שני מפעל של אירועים הוא מפעל ה אחראי על קבלת מידע ו על אירועים ו אחראי על יצורם

מכל סוג ו גם הוא מתפל בשגאות קלט

*נוכל לראות את זה גם בתרשים uml

eventFactory



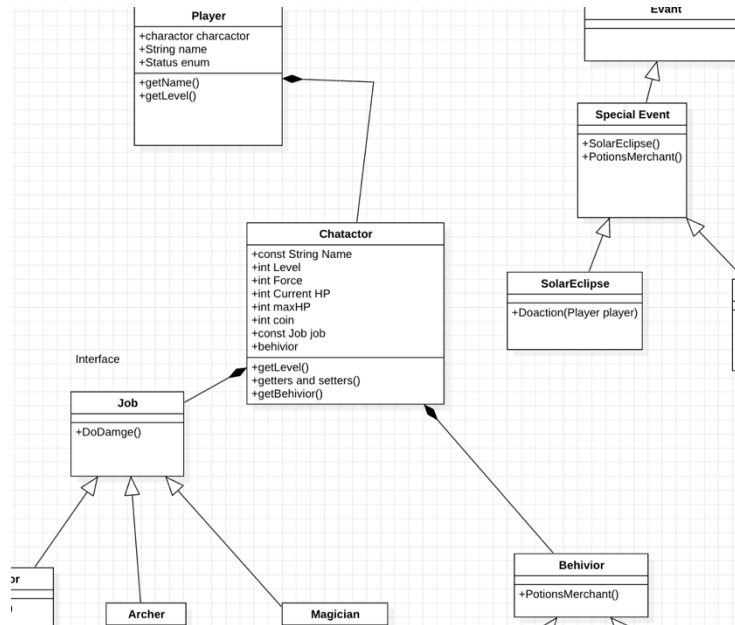
נעבור ל סוגי **Structural patterns**:

מסוג זה בעיקר השתמשנו ב **composite** והוא הכי נפוץ כי מדובר על יחס בעלות

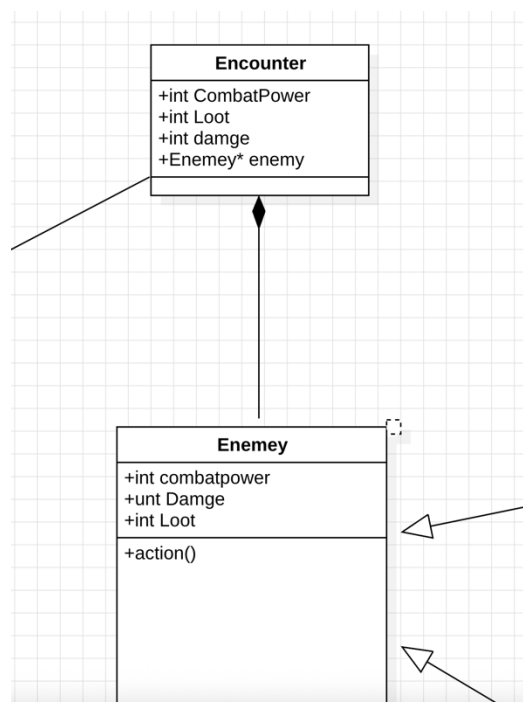
כמה דוגמאות מ עיצוב שלנו

השתמשנו ב **composite** הרבה כי כך

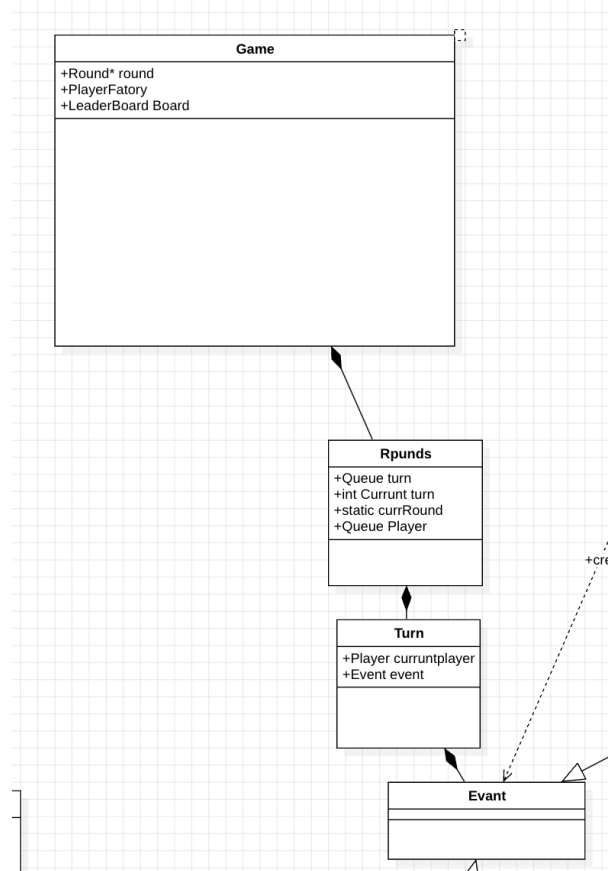
*נוכל לראות ש שחקן הוא יש לו בעלות ל אובייקט מסוג character
 * כל character יש לו בעלות על איביקט מסוג behavior
 * גם כל character יש לו בעלות מחלקה job



* כל אירוע Encounter יש לו בעלות על אובייקט מסוג enemy



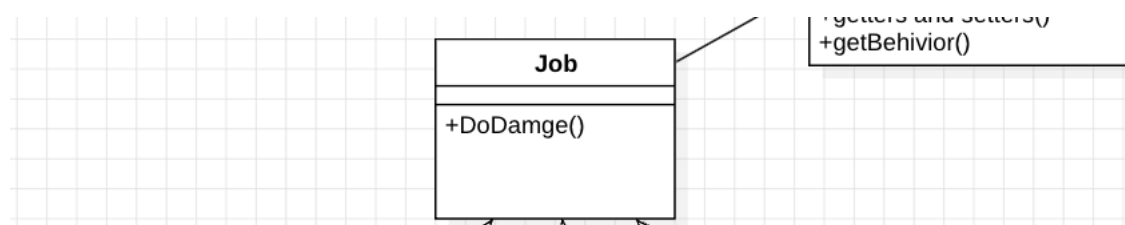
- כל משחק יש לו בעלות על אובייקט Round
- כל Round יש לו בעלות על אובייקט turn
- כל turn יש לו בעלות על אירוע

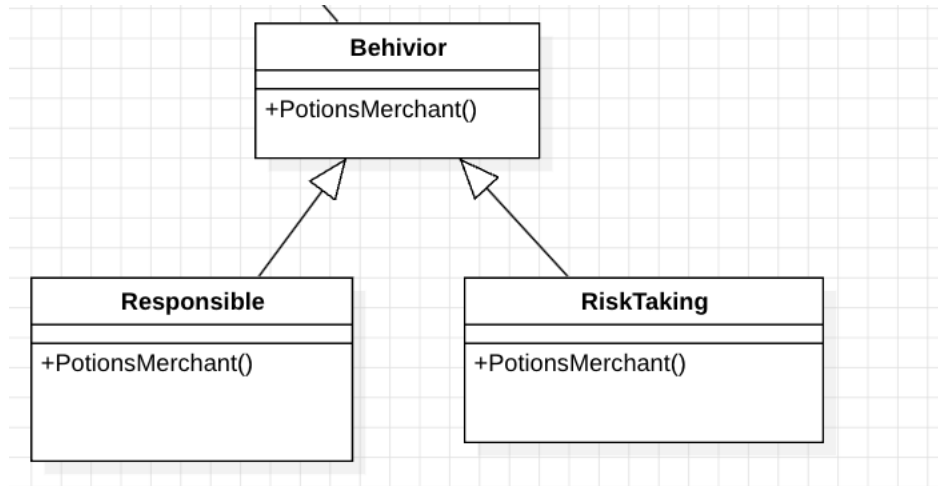


נעבור לסוג אחרון והוא Behavioral patterns:

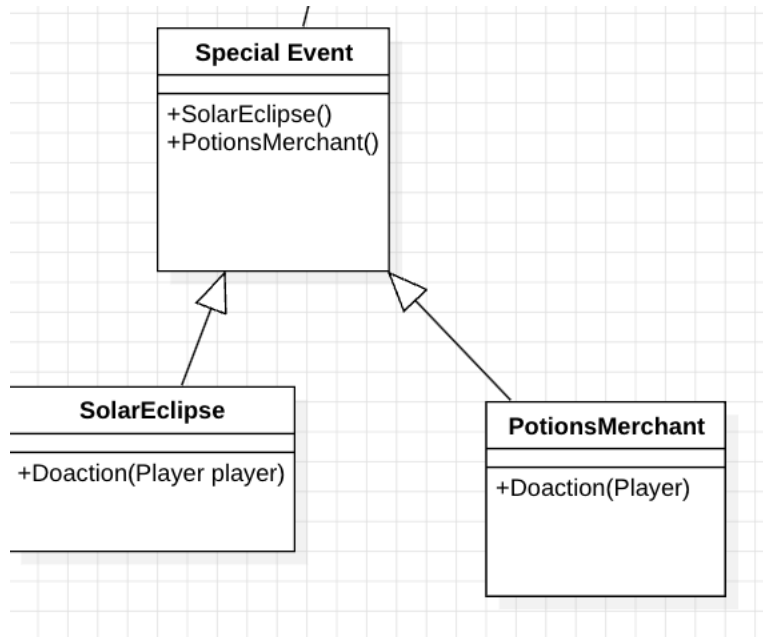
השתמשנו ב סוג **stratgy** וזהו למעשה לב המשחק – לאפשר פולימורפיזם התנהגותי, שבו לכל אובייקט (כגון שחקן עם מקצוע או התנהגות שונה) יש תגובה שונה לאותו אירוע. תבנית ה־ Strategy מאפשרת לנו להחליף את ההתנהגות הדינמית מבלי לשנות את מבנה המחלקות, וכך לממש קוד גמיש וניתן להרחבה.

*יש שלושה סוגים של עבודה כל אחד יורש מ abstract job



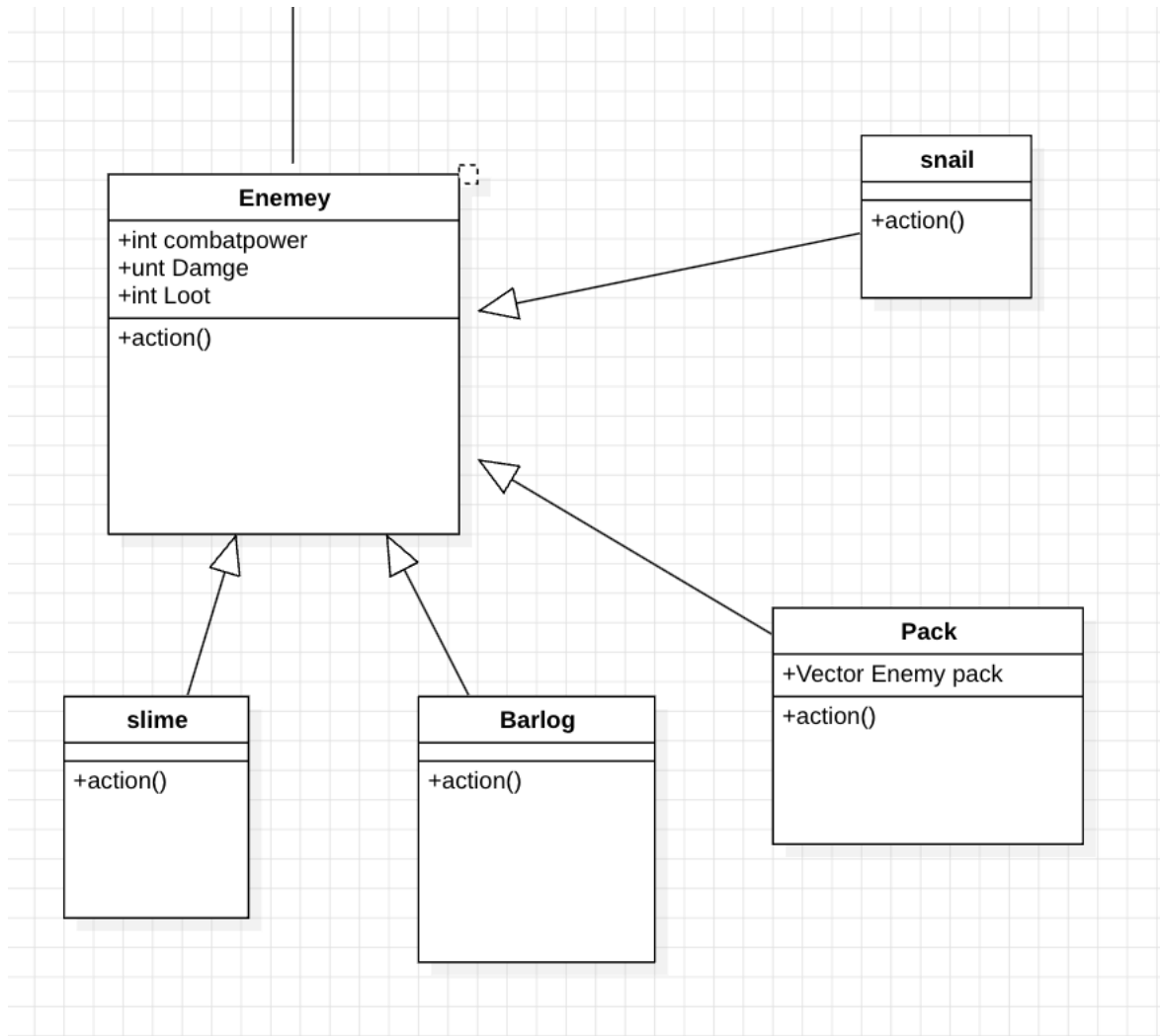


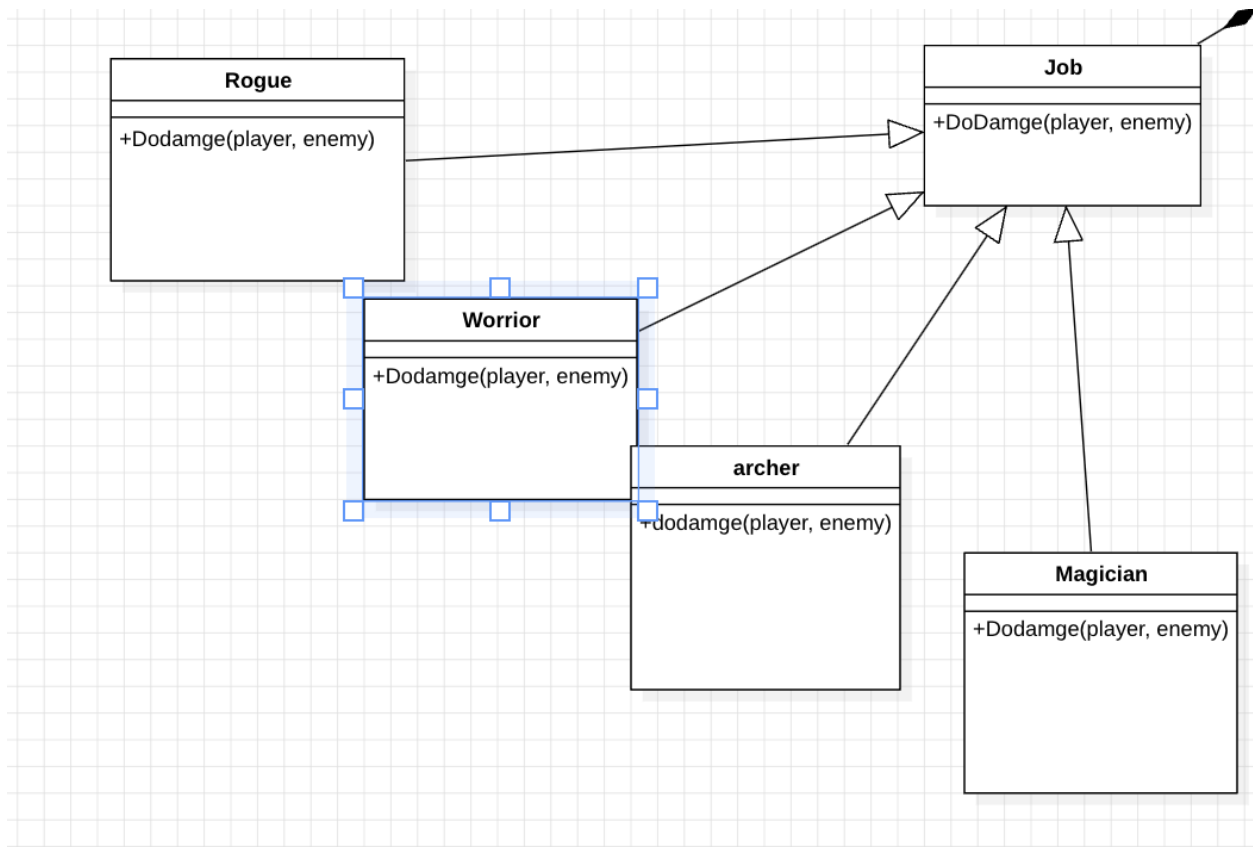
*יש שני סוגים של אירועים מיוחדים וכל אחד יורש ו עושה השפעה אחרת



Enemy is an abstract class*

יש 4 סוגי enemy שונים כל אחד מתנהג בצורה שונה
לאותה פונקציה





*כיוון ש ההתקלות תלויה אך ו רק ב אירוע Encounter ו אם יכולת קרב ל enemy של האירוע גדול פי2

*אני רק אעשה דריסה ל פונקציית ה virtual Dodamge(std::shared_ptr<enemy>enemy) ב
 מחלקת job ו אממש אותה כמו שאני רוצה אבדוק אם היכולת הקרב של enemy גדולה פי 2

```

class Enemy;
class Encounter;
class Character;
class Job {
public:
    Job () ;
    ~Job() = default ;
    virtual void doJob(Character& character); // nothing
    virtual const std::string getJob() const = 0;
    virtual void SolarEclipse(Character& character) ;
    virtual void applyin(std::shared_ptr<Player>player)=0;
    virtual void applyin(std::shared_ptr<Player>player,std::shared_ptr<Enemy>enemy)=0;
    static std::shared_ptr<Job> fromString(const std::string &name) ;
};

```

מ של character ש ב player שמקבל אותו
 *וגם צריך להוסיף תנאי חדש ב פונקציה fromString שמתבסס עליה ה player factory

```

std::shared_ptr<Job> Job::fromString(const std::string &name) {
    if (name == "Warrior") {return std::make_shared<Warrior>();}
    if (name == "Magician") { return std::make_shared<Magician>();}
    if ( name == "Archer") { return std::make_shared<Archer>();}
    if ( name == "Rogue") { return std::make_shared<Archer>();}

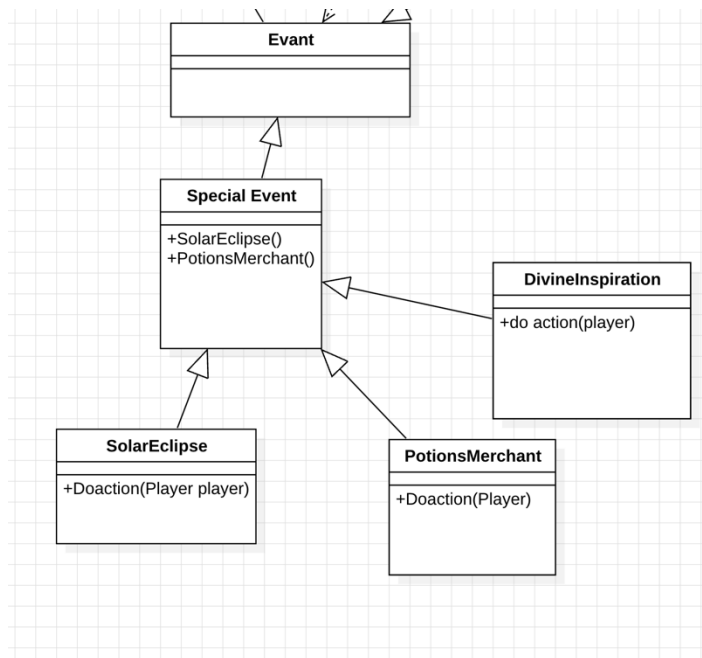
    throw std::invalid_argument(s:"Unknown job: " + name);
}

```

- זה מראה כמה חזקה הפולימורפיזם התנהגותי כי אני לא מסתבך מי קורה לה וכל בלגן זה פשוט אני יש לי בסיס ש עובד יש לו polymorphism behavior אני רק דורס ו יש לי ההתנהגות ו job חדש

(4) הוספת אירוע חדש מסוג DivineInspiration :

כן אפשר להוסיף אירוע חדש שמתנהג כך קודם כל נגדיר מחלקה שלו שיורשת מ מחלקת Event



```
#pragma once
#include <memory>
#include <string>

class Character;

class Player;
class Event : public std::enable_shared_from_this<Event> {
    friend class Character;
    friend class Player;
protected:
    std::string name;
    Event() = default;
    //virtual void applyEvent( std::shared_ptr<Character> character):
    virtual void applyEventP( std::shared_ptr<Player> player) = 0;

public:
    Event(const std::string &name);
    std::string getName() const;
    virtual std::string getDescription() const = 0;
};
```

**))

נדרוש

```
#pragma once
#include ...
class Event;
class Job;

class Character : public std::enable_shared_from_this<Character>{
    friend class Warrior ;
    friend class Archer ;
    friend class Magician ;
    friend class job;
friend class Event;
    friend class DivineInspiration;
    friend class Behavior;
    class Enemy;
    const std::string name ;
    int Level;
    int CurrentHP;
    int maxHP;
    const std::shared_ptr< Job> job;
    const std::shared_ptr<Behavior> behavior ;
    int coin ;
    int force ;
public:
```

```
int maxHP;
std::shared_ptr< Job> job;
```

נוסיר את ה const

*נוסיר private setter for the job

```
int coin ;
int force ;
void setJob(const std::shared_ptr<Job>& newJob) {
    this->job = newJob;
}
public:
```

*נגדיר במחלקה DivineInspiration :

* vector שמכיל את shared pointer של jobs שונים כמו warrior, archer, magician

ובפונקציה applyEventP מקודם**) בתוך הפונקציה נשתמש ב random שנותנת לנו מספר בין 0 ל 2
נשתמש ב [] operator של vector בתוכו מספר רנדומלי ו עכשיו נעביר ל פונקציה זו

```
class Job {
public:
    Job () ;
    ~Job() = default ;
    virtual void doJob(Character& character); // nothing
    virtual const std::string getJob() const = 0;
    virtual void SolarEclipse(Character& character) ;
    virtual void applyIn(std::shared_ptr<Player>player)=0;
    virtual void applyIn(std::shared_ptr<Player>player, std::shared_ptr<Enemy>enemy)=0;
    void applyDivineInspiration(std::shared_ptr<Player>player, std::shared_ptr<Job>randomJob);
    static std::shared_ptr<Job> fromString(const std::string &name) ;
};
```

כיון שהיא לא תלויה ב job האחרון נוכל רק להגדיר אותה רק job

* ובתוך פונקציה זו נשתמש ב jobsetter ל job הרנדומלי שנעבר של ה character של שחקן

*כיוון ש job הוא abstract class הנתונים של השחקן לא נשמרות בו כל השפעות היו לו נשמרו

